

우리 기수는 유니티에서 다음과 같은 코드 스타일을 갖고 있어, 이 부분을 앞으로 코드를 짜는 데 참고해줘

- 멤버변수는 `_aaa` 이고 소문자로 시작
- 유니티에서 참조하는 객체는 대문자로 시작 `Text_AAA;`
- 유니티에서 참조하는 객체는 기본적으로 `[SerializeField]`에 `private`
- 코드 내에서 다른 객체가 참조해서 프로퍼티로 열어야하는 상황에는 `public`보다는 `AAA{get;set;} 사용`
- 기본 `private` 원칙이지만 필요한 경우 `public Get, Set` 함수를 추가로 제공하기도 함
- 클래스 범위 내에서만 사용하는 메서드들은 `private` 원칙
- 클래스 밖에서 다른 객체가 부르는 경우는 `public`
- 함수는 동사로 시작할 것
-
- 그 외 매니저 규칙
- 게임의 전체적인 흐름이나 진행 로직은 `GameManager`를 통함
- 단 내가 제시한 장르가 `GameManager`의 세이브 로드 외에 부가적인 독립적인 룰이 있다면 `BattleManager, DeckManager, TurnManager` 같은 매니저를 추가로 두도록 함
- 다만, 기본적인 인스턴스 데이터(게임 진행 중 저장해야하는 데이터들은 모두 `Model`이름을 뒤에 붙여서 데이터 모델을 만들어두고 이를 `GameManager`안에 소유하고 있는 구조 -> 예시 `PlayerModel`)
- UI의 관리는 `UIManager`를 통함
- `SoundManager`는 사운드 재생 관리
- `GameDataManager`는 `static` 데이터 `Json`으로 불러와서 `GameData` 클래스에 매핑해서 사용함
- `ResourceManager`는 `Addressable` 기반을 많이 쓰나, 필요에 따라 `Resource.Load`를 사용하는 경우도 있음, 그러니 프리팹이나 게임 오브젝트 같이 동기적 처리가 필요한 경우가 아니라면 `Addressable`로 비동기 로딩을 하고 있다
- `UIManager`에는 핵심 오픈 클로즈 로직이 있지만, 추가하는 콘텐츠들은 `UIManagerExtension`이라는 확장메서드 클래스에서만 추가하고 있다
- 저장에 대한 핵심 로직은 `NetworkManager`안에 들어가 있다
- 공용으로 특히 데이터를 가공하는 유형의 동작은 `GameUtil`이라는 `static class`의 `static` 메서드에서 반환형과 파라미터를 사용해서 순수 결과 값만 받아오는 구조
- 마지막으로 게임 오브젝트 매니저는 씬에서 동적생성할 거의 모든 게임 오브젝트를 만들고, 제거하는 역할을 하며, 이는 `int`범위 내에서 `generate`된 인스턴스 `id`를 가져, 즉 몬스터들이 충돌되고 그 몬스터 객체가 이미 인스턴스 `id`를 생성될때 갖고 있으므로, 자신의 정보를 수정할 때 `GameObjectManager`를 통하도록 해줘
- 우선 기본적인 큰 틀은 다음과 같고 연관 코드는 다음과 같아

```
using System.Collections.Generic;
using UnityEngine;
```

```

public class DaniTechGameManager : MonoBehaviour
{
    public static DaniTechGameManager Inst { get; set; }

    // 플레이 중에 저장되어야 하는 정보들이 있는 위치
    private DaniTechPlayerModel _playerModel = new DaniTechPlayerModel();

    private void Awake()
    {
        Inst = this;
    }

    private void Start()
    {
        LoadSaveData();
    }

    public void SaveData()
    {
        DaniTechNetworkManager.Inst.RequestSaveData(_playerModel);
    }

    public void SaveAndEndGame()
    {
        SaveData();
        Application.Quit();
    }

    private void LoadSaveData()
    {
        _playerModel = DaniTechNetworkManager.Inst.RequestLoadSaveData();
    }

    public void IncreasePlayerExp(int exp)
    {
        // 추후에 한곳에서 관리할 수 있게 익스텐션으로 빼도 된다
        _playerModel.PlayerTotalExp += exp;
    }

    public void AddItem(string itemDataId, int addItemCount)
    {
        // 저장할때 고유값 ID를 부여하기 위해 사용
        long uniqueId = DaniTechGameUtil.GenerateUniqueId();

        // TODO : 우선 쉽게 사용할 수 있도록 중복 처리는 빼두었다. 습득할때마다 아이템
        이 하나씩 추가되도록 해두고
    }
}

```

```

        // 추후에 중복값은 StackCount가 다 찰때까지 누적해줄 수 있도록 로직을 추가하자
        var newItem = new DaniTechItemModel();
        newItem.ItemUniqueId = uniqueId;
        newItem.ItemDataId = itemDataId;
        newItem.ItemStackCount = addItemCount;

        _playerModel.ItemList.Add(newItem);
    }

    public List<DaniTechItemModel> GetPlayerItemList()
    {
        // _playerModel이 Private이므로 외부에서 ItemList를 받아올 수 있게 Get함수를
        // 사용한다
        return _playerModel.ItemList;
    }
}

```

```

using Cysharp.Threading.Tasks;
using System.Collections.Generic;
using UnityEngine;

public class DaniTechGameObjectManager : MonoBehaviour
{
    // 생성할 몬스터의 프리팹
    [SerializeField] private GameObject Prefab_Enemy;
    [SerializeField] private Transform Root_Enemy;

    public static DaniTechGameObjectManager Inst { get; set; }

    // 생성된 오브젝트의 키가 됨
    private int _objectInstanceKeyGenerator = 0;

    // 생성된 오브젝트의 생명을 보관
    private Dictionary<int, GameObject> _createdGameObjectContainer = new
    Dictionary<int, GameObject>();
    private Dictionary<int, DaniTech_2DFieldObject> _fieldObjectContainer =
    new Dictionary<int, DaniTech_2DFieldObject>();

    private void Awake()
    {
        Inst = this;
    }

    public void RequestSpawnEnemy()
    {
    }
}

```

```

{
    if(Prefab_Enemy == null)
    {
        Debug.LogWarning("프리팸이 등록되지 않은 오브젝트 입니다.");
        return;
    }

    var gObj = Instantiate(Prefab_Enemy, Root_Enemy);
    if(gObj == null)
    {
        Debug.LogWarning("생성에 실패한 게임 오브젝트 입니다.");
        return;
    }

    // 1-1 생성에 성공했다면, 미리 Key를 발급한다.
    _objectInstanceKeyGenerator++;

    // 1-2 Dictionary에 추가하기 전에 미리 키 검사한다
    if
(_createdGameObjectContainer.ContainsKey(_objectInstanceKeyGenerator) == true)
    {
        Debug.LogWarning("이미 동일한 키가 발급된 게임 오브젝트가 존재합니다");
        return;
    }

    // 1-3 동적생성(실체화)된 오브젝트를 게임 오브젝트 매니저의 자료구조
(Dictionary)에 보관하자!
    _createdGameObjectContainer.Add(_objectInstanceKeyGenerator, gObj);
    InitGeneratedEntityObject(_objectInstanceKeyGenerator, gObj);

    Debug.Log($"키: {_objectInstanceKeyGenerator}의 객체 {gObj.name}이 호출되
었습니다.");
}

private void InitGeneratedEntityObject(int generatedId, GameObject gObj)
{
    // 4-1 지금은 Enemy지만, 나중에 IGameEntity 같은 인터페이스로 개선하면 더 좋
다
    DaniTech_2DEnemy gameEntity = gObj.GetComponent<DaniTech_2DEnemy>();
    if(gameEntity == null)
    {
        Debug.LogWarning($"생성된 {gObj.name}의 InstanceId를 대입할 수 있는 컴
포넌트를 가져올 수 없습니다!");
        return;
    }
}

```

```

        // 4-2 생성된 객체에 정보를 부여한다!
        gameEntity.InitEnemyInfo(generatedId);
    }

    public GameObject GetEntityObjectCanBeNull(int instanceId)
    {
        if(_createdGameObjectContainer.ContainsKey(instanceId) == false)
        {
            Debug.LogWarning($"{instanceId}는 존재하지 않습니다.");
            return null;
        }

        // 2-1 실체화하면서 등록된 게임 오브젝트가 있다면 반환
        return _createdGameObjectContainer[instanceId];
    }

    public void RequestDestroyEntityObject(int instanceId)
    {
        var gObj = GetEntityObjectCanBeNull(instanceId);
        if(gObj == null)
        {
            return;
        }

        // 3-1 요청된 객체를 제거함
        _createdGameObjectContainer.Remove(instanceId);
        Destroy(gObj);
    }

```

//[필드 오브젝트]

```

=====
=====

```

```

    public async UniTaskVoid CreateFieldObject(string fieldObjectDataId,
    Transform spawnSpot)
    {
        var fieldObject =
    DaniTechGameDataManager.Instance.GetDNFieldObjectData(fieldObjectDataId);
        if (fieldObject != null)
        {

```

```

        var createdObj = await
DaniTechResourceManager.Inst.InstantiateAsync(fieldObject.PrefabPath,
Root_Enemy, true);
        createdObj.transform.position = spawnSpot.position;
        AddFieldObjectOnCreate(createdObj, fieldObjectDataId);
    }
}

private void AddFieldObjectOnCreate(GameObject createdObject, string
fieldObjectDataId)
{
    _objectInstanceKeyGenerator++;
    var generatedInstanceId = _objectInstanceKeyGenerator;
    var fieldObject = createdObject.GetComponent<DaniTech_2DFieldObject>
();

    if(fieldObject != null)
    {
        _fieldObjectContainer.Add(generatedInstanceId, fieldObject);
        fieldObject.InitFieldObjectInfoOnCreated(generatedInstanceId,
fieldObjectDataId);
    }
}

public void RequestDestroyFieldObject(int instanceId)
{
    var fieldObjectComponent = GetFieldObjectById(instanceId);
    if (fieldObjectComponent == null)
    {
        return;
    }

    // 요청된 필드 오브젝트를 제거함
    _fieldObjectContainer.Remove(instanceId);
    Destroy(fieldObjectComponent.gameObject);
}

public DaniTech_2DFieldObject GetFieldObjectById(int
fieldObjectInstanceId)
{
    if(_fieldObjectContainer.ContainsKey(fieldObjectInstanceId) == false)
    {
        Debug.LogError($"{fieldObjectInstanceId} 찾으려는 필드 오브젝트가 유효
하지 않습니다");
        return null;
    }
}

```

```

        return _fieldObjectContainer[fieldObjectId];
    }
}

```

```

using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using UnityEngine;

public class DaniTechGameDataManager : MonoBehaviour
{
    public static DaniTechGameDataManager Instance { get; set; }

    private void Awake()
    {
        Instance = this;

        // +++ C# 콘솔때와 다르게 이제 Main()함수가 아닌
        // 모노의 메서드에서 호출될 수 있으므로, 데이터 매니저가 활성화되면 바로 모든 데
        이터를 한번 받아오자
        // 이처리는 원하는 시점이 있다면 이전해도 된다
        DaniTechGameUtil.LoadFullData();
    }

    // --- JsonUtility의 한계를 극복하기 위한 Wrapper 클래스 ---
    [Serializable]
    private class SerializationWrapper<T>
    {
        public List<T> items; // JSON 파일의 루트 키 이름이 "items"여야 함
    }
    // -----

    public Dictionary<string, DNCharacterData> CharacterDataList { get;
private set; } = new Dictionary<string, DNCharacterData>();
    public Dictionary<string, DNSkillData> SkillDataList { get; private set; }
= new Dictionary<string, DNSkillData>();
    public Dictionary<string, DNWeaponData> WeaponDataList { get; private set;
} = new Dictionary<string, DNWeaponData>();
    public Dictionary<string, DNCostumeData> CostumeDataList { get; private
set; } = new Dictionary<string, DNCostumeData>();
    public Dictionary<string, DNItemData> ItemDataList { get; private set; } =
new Dictionary<string, DNItemData>();

```

```

    public Dictionary<string, DNDDialogueGroupData> DialogueGroupDataList {
get; private set; } = new Dictionary<string, DNDDialogueGroupData>();
    public Dictionary<string, DNDDialogueData> DialogueDataList { get; private
set; } = new Dictionary<string, DNDDialogueData>();
    public Dictionary<string, DNFieldObjectData> FieldObjectDataList { get;
private set; } = new Dictionary<string, DNFieldObjectData>();
    public Dictionary<string, DNMonsterData> MonsterDataList { get; private
set; } = new Dictionary<string, DNMonsterData>();

    private Dictionary<string, T> LoadData<T>(string tableName) where T :
GameDataBase
    {
        // 1. 경로 설정 (확장자 .json 제외!)
        // Resources/JsonOutput 폴더
        string resourcePath = $"JsonOutput/{tableName}";

        // 2. 리소스 로드
        TextAsset textAsset = Resources.Load<TextAsset>(resourcePath);

        // 3. 파일 존재 여부 체크
        if (textAsset == null)
        {
            Debug.LogError($"[Error] 리소스를 찾을 수 없습니다:
Resources/{resourcePath}");
            return new Dictionary<string, T>();
        }

        try
        {
            string jsonString = textAsset.text;

            // 4. JsonUtility용 Wrapper 트릭 적용
            string wrappedJson = "{\"items\":" + jsonString + "}";
            SerializationWrapper<T> wrapper =
JsonUtility.FromJson<SerializationWrapper<T>>(wrappedJson);

            if (wrapper != null && wrapper.items != null)
            {
                Debug.Log($"typeof(T).Name} 데이터를 {wrapper.items.Count}개 로
드했습니다.");
                // ToDictionary를 사용하려면 각 클래스(T)에 Id 필드가 있어야 합니다.
                return wrapper.items.ToDictionary(item => item.Id.ToString());
            }
        }
        catch (Exception ex)
        {

```



```

        Debug.LogError($"[{typeof(T).Name} JSON 로드 오류] {ex.Message}");
    }

    return new Dictionary<string, T>();
}

public void LoadSkillData(string jsonPath)
{
    SkillDataList = LoadData<DNSkillData>(jsonPath);
}

public void LoadCharacterData(string jsonPath)
{
    CharacterDataList = LoadData<DNCharacterData>(jsonPath);
}

public void LoadWeaponData(string jsonPath)
{
    WeaponDataList = LoadData<DNWeaponData>(jsonPath);
}

public void LoadCostumeData(string jsonPath)
{
    CostumeDataList = LoadData<DNCostumeData>(jsonPath);
}

public void LoadDNItemData(string jsonPath)
{
    ItemDataList = LoadData<DNItemData>(jsonPath);
}

public void LoadDNDialogueData()
{
    DialogueGroupDataList = LoadData<DNDialogueGroupData>
("DNDialogueGroup");
    DialogueDataList = LoadData<DNDialogueData>("DNDialogue");
}

public void LoadAll()
{
    FieldObjectDataList = LoadData<DNFieldObjectData>("DNFieldObject");
    MonsterDataList = LoadData<DNMonsterData>("DNMonster");
}

// [아래는 사용을 위한 부분들을 메서드 정의]

```

```
=====
=====
// Get과 Find이름을 꼭 구별 하자!

public DNCharacterData GetCharacterData(string id)
{
    if (CharacterDataList == null || string.IsNullOrEmpty(id)) return
null;

    return CharacterDataList.TryGetValue(id, out var item) ? item : null;
}

public DNSkillData GetSkill(string id)
{
    if (SkillDataList == null || string.IsNullOrEmpty(id)) return null;

    return SkillDataList.TryGetValue(id, out var item) ? item : null;
}

public DNWeaponData GetWeaponData(string id)
{
    if (WeaponDataList == null || string.IsNullOrEmpty(id)) return null;

    return WeaponDataList.TryGetValue(id, out var data) ? data : null;
}

public DNCostumeData GetCostumeData(string id)
{
    if (CostumeDataList == null || string.IsNullOrEmpty(id)) return null;

    return CostumeDataList.TryGetValue(id, out var data) ? data : null;
}

public DNItemData GetDNItemData(string id)
{
    if (ItemDataList == null || string.IsNullOrEmpty(id)) return null;

    return ItemDataList.TryGetValue(id, out var data) ? data : null;
}

public DNDialogueGroupData GetDNDialogueGroupData(string dataId)
{
    if (DialogueGroupDataList == null || string.IsNullOrEmpty(dataId))
return null;

    return DialogueGroupDataList.TryGetValue(dataId, out var data)
```

```

: null;
    }

    public DNDialogueData GetDNDialogueData(string dataId)
    {
        if (DialogueDataList == null || string.IsNullOrEmpty(dataId)) return
null;

        return DialogueDataList.TryGetValue(dataId, out var data) ? data :
null;
    }

    public DNMonsterData GetDNMonsterData(string dataId)
    {
        if (MonsterDataList == null || string.IsNullOrEmpty(dataId)) return
null;

        return MonsterDataList.TryGetValue(dataId, out var data) ? data :
null;
    }

    public DNFieldObjectData GetDNFieldObjectData(string dataId)
    {
        if (FieldObjectDataList == null || string.IsNullOrEmpty(dataId))
return null;

        return FieldObjectDataList.TryGetValue(dataId, out var data) ? data :
null;
    }
}

```

```

using Cysharp.Threading.Tasks;
using System;
using System.Collections.Generic;
using System.IO;
using System.Threading;
using UnityEngine;
using UnityEngine.UI;

public static class DaniTechGameUtil
{
    // 마지막으로 할당된 ID를 전역적으로 기록 (스레드 안전)
    private static long _lastId = 0;

    public static void LoadFullData()

```

```

{
    // 게임 로딩할 때 불러올 데이터는 여기서!
    DaniTechGameDataManager.Instance.LoadSkillData("Skill");
    DaniTechGameDataManager.Instance.LoadCharacterData("Character");
    DaniTechGameDataManager.Instance.LoadWeaponData("Weapon");
    DaniTechGameDataManager.Instance.LoadCostumeData("Costume");
    DaniTechGameDataManager.Instance.LoadDNItemData("DNItem");
    DaniTechGameDataManager.Instance.LoadDNDialogueData();
    DaniTechGameDataManager.Instance.LoadAll();
}

public static int CalcCharacterFinalDamage(int curCharacterLevel, int
levelPerDamage, bool isCritical)
{
    int damagePerLevel = (curCharacterLevel + levelPerDamage);
    int finalDamage = isCritical ? (damagePerLevel * 2) : damagePerLevel;
    return finalDamage;
}

public static Sprite LoadSpriteCanBeNull(string spriteName)
{
    // 1. Resources/ 경로에서 이름으로 스프라이트 로드
    // 예: spriteName이 "Sword"라면 Assets/Resources/2D/Sword.png를 찾음
    // 이 2D같은 경로는 나중에 Sprite, Texture 등등 다양하게 바뀌어도 무관합니다!
    Sprite loadedSprite = Resources.Load<Sprite>($" {spriteName}");

    if (loadedSprite != null)
    {
        return loadedSprite;
    }

    Debug.LogError($"에셋을 찾을 수 없습니다: {spriteName}");
    return null;
}

public static async UniTask<Sprite> LoadAndSetSpriteImage(Image
targetImage, string spritePath)
{
    Sprite sprite = await
DaniTechResourceManager.Inst.LoadSprite(spritePath);
    if (sprite != null)
    {
        targetImage.sprite = sprite;
    }
    return sprite;
}

```

```

        public static async UniTaskVoid LoadAndPlayAudioClip(AudioSource
audioSource, string audioPath, bool isLoop = false)
        {
            AudioClip clip = await
DaniTechResourceManager.Inst.LoadAsset<AudioClip>(audioPath);
            if (clip == null)
            {
                Debug.LogError($"{audioPath}를 찾을 수 없습니다! 어드레서블 설정이 되어
있는지 확인해주세요.");
                return;
            }

            if(isLoop == true)
            {
                audioSource.clip = clip;
                audioSource.loop = true;
                audioSource.Play();
            }
            else
            {
                audioSource.PlayOneShot(clip);
            }
        }

        public static async UniTaskVoid LoadAndSetTexture(RawImage targetRawImage,
string texturePath)
        {
            // 비동기로 로드하기 전까지는 해당 오브젝트를 잠깐 비활성화 해준다
            targetRawImage.gameObject.SetActive(false);
            Texture texture = await
DaniTechResourceManager.Inst.LoadAsset<Texture>(texturePath);
            if (texture != null)
            {
                targetRawImage.texture = texture;
            }
            targetRawImage.gameObject.SetActive(true);
        }

        public static List<string> GetDialogueIdList(string dialogueGroupId)
        {
            var list = new List<string>();

            // "dialogue_group_mainstream_1_1"
            var data =
DaniTechGameDataManager.Instance.GetDNDDialogueGroupData(dialogueGroupId);

```

```

        if (data != null)
        {
            var idArr = data.DialogueIdList;
            foreach(var id in idArr)
            {
                list.Add(id);
            }
        }

        return list;
    }

    // 그냥 유니크 키가 발급되어야 할 때 사용하려고 만든 것 (의미가 있는 건 아니므로 사
    용만 하세요)
    public static long GenerateUniqueId()
    {
        long newId = DateTime.UtcNow.Ticks;

        // 원자적 연산으로 안전하게 ID 갱신
        while (true)
        {
            long lastId = Volatile.Read(ref _lastId);

            // 만약 현재 시간이 이전 ID보다 작거나 같다면 (루프가 너무 빠른 경우 포함)
            // 이전 ID + 1로 강제 설정하여 중복 방지
            long idToAssign = (newId <= lastId) ? lastId + 1 : newId;

            // _lastId가 내가 읽은 시점과 같다면 idToAssign으로 교체 (성공 시 루프
            탈출)
            if (Interlocked.CompareExchange(ref _lastId, idToAssign, lastId)
            == lastId)
            {
                return idToAssign;
            }
            // 그 사이 다른 스레드가 값을 바꿨다면 다시 시도
        }
    }
}

```

```

using System.Collections.Generic;
using System.IO;
using UnityEngine;

```

```

public class DaniTechNetworkManager : MonoBehaviour

```

```

{
    public static DaniTechNetworkManager Inst { get; set; }

    private void Awake()
    {
        Inst = this;
    }

    // 파일 저장 경로 설정 (C:/Users/이름/.../projectName/save.json)
    private string GetPath()
    {
        return Path.Combine(Application.persistentDataPath,
"DaniTechSaveData.json");
    }

    // 세이브 기능 구현
    public void RequestSaveData(DaniTechPlayerModel data)
    {
        // prettyPrint = true는 JSON을 보기 좋게 정렬
        string json = JsonUtility.ToJson(data, true);
        File.WriteAllText(GetPath(), json); // 파일 쓰기는 상당한 비용이 소모됨!
        Debug.Log($"저장 완료: {GetPath()}");
    }

    // 로드 기능
    public DaniTechPlayerModel RequestLoadSaveData()
    {
        string path = GetPath();
        if (File.Exists(path))
        {
            string json = File.ReadAllText(path);
            DaniTechPlayerModel data =
JsonUtility.FromJson<DaniTechPlayerModel>(json);
            Debug.Log("데이터를 불러왔습니다.");
            return data;
        }
        else
        {
            Debug.LogWarning("세이브 파일이 없습니다. 새 데이터를 생성합니다.");
            var playerData = GetDefaultPlayerData();
            RequestSaveData(GetDefaultPlayerData());
            return playerData;
        }
    }

    public DaniTechPlayerModel GetDefaultPlayerData()

```

```

    {
        var newPlayerData = new DaniTechPlayerModel();
        newPlayerData.PlayerName = "NoName";
        newPlayerData.PlayerTotalExp = 0;
        return newPlayerData;
    }
}

```

```

using Cysharp.Threading.Tasks;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.AddressableAssets;
using UnityEngine.ResourceManagement.AsyncOperations;

public class DaniTechResourceManager : MonoBehaviour
{
    public static DaniTechResourceManager Inst { get; set; }

    private void Awake()
    {
        Inst = this;
    }

    // 로드된 에셋들을 관리하기 위한 캐시 (메모리 해제 시 필요)
    private Dictionary<string, AsyncOperationHandle> _handles = new
Dictionary<string, AsyncOperationHandle>();

    //// 1. 에셋 로드 함수 (제네릭 사용)
    public void LoadAsset<T>(string address, System.Action<T> callback) where
T : UnityEngine.Object
    {
        // 이미 로드된 에셋인지 확인
        if (_handles.TryGetValue(address, out AsyncOperationHandle handle))
        {
            callback?.Invoke(handle.Result as T);
            return;
        }

        // 어드레서블 로드 실행
        AsyncOperationHandle<T> loadHandle = Addressables.LoadAssetAsync<T>
(address);

        // 비동기 처리를 위한 한시적 람다 사용
        loadHandle.Completed += (op) =>

```



```

        {
            if (op.Status == AsyncOperationStatus.Succeeded)
            {
                _handles[address] = op; // 핸들 저장
                callback?.Invoke(op.Result);
            }
            else
            {
                Debug.LogError($"에셋 로드 실패: {address}");
            }
        };
    }

    public async UniTask<T> LoadAsset<T>(string address) where T :
    UnityEngine.Object
    {
        // 1. 이미 로드된 에셋인지 확인 (캐싱 확인)
        if (_handles.TryGetValue(address, out AsyncOperationHandle handle))
        {
            // 이미 완료된 핸들이라면 즉시 결과 반환
            return handle.Result as T;
        }

        // 2. 어드레서블 로드 실행 (UniTask로 변환)
        // ToUniTask()를 사용하여 await 가능하게 만듭니다.
        AsyncOperationHandle<T> loadHandle = Addressables.LoadAssetAsync<T>
(address);

        try
        {
            T result = await loadHandle.ToUniTask();

            // 3. 핸들 저장 (성공 시)
            _handles[address] = loadHandle;
            return result;
        }
        catch (System.Exception e)
        {
            // 4. 실패 시 예외 처리
            Debug.LogError($"에셋 로드 실패: {address} / Error: {e.Message}");

            // 실패한 핸들도 메모리 해제가 필요할 수 있으므로 상황에 따라 Release 처리
            if (loadHandle.IsValid())
                Addressables.Release(loadHandle);

            return null;
        }
    }

```

```

    }
}

public async UniTask<GameObject> InstantiateAsync(string address,
Transform parent = null, bool instantiateInWorldSpace = false)
{
    // 1. 생성 시도
    AsyncOperationHandle<GameObject> handle =
Addressables.InstantiateAsync(address, parent, instantiateInWorldSpace);

    try
    {
        // 2. UniTask로 변환하여 비동기 대기
        GameObject instance = await handle.ToUniTask();

        // 3. 성공 시 생성된 오브젝트 반환
        return instance;
    }
    catch (System.Exception e)
    {
        // 4. 실패 시 예외 처리 및 핸들 해제
        Debug.LogError($"프리팹 생성 실패: {address} / Error: {e.Message}");

        if (handle.IsValid())
            Addressables.Release(handle);

        return null;
    }
}

// 2-1. 스프라이트 로드 함수
public void LoadSprite(string address, System.Action<Sprite> callback)
{
    // 이미 로드된 스프라이트인지 확인 (캐시 활용)
    if (_handles.TryGetValue(address, out AsyncOperationHandle handle))
    {
        callback?.Invoke(handle.Result as Sprite);
        return;
    }

    // 스프라이트 형식으로 로드
    AsyncOperationHandle<Sprite> handleOrigin =
Addressables.LoadAssetAsync<Sprite>(address);

    handleOrigin.Completed += (op) =>
    {

```

```

        if (op.Status == AsyncOperationStatus.Succeeded)
        {
            _handles[address] = op; // 핸들 저장 (나중에 Release하기 위함)
            callback?.Invoke(op.Result);
        }
        else
        {
            Debug.LogError($"스프라이트 로드 실패: {address}");
        }
    };
}

public async UniTask<Sprite> LoadSprite(string address)
{
    // 1. 이미 로드된 스프라이트인지 확인 (캐시 활용)
    if (_handles.TryGetValue(address, out AsyncOperationHandle handle))
    {
        // 결과가 Sprite인지 확인 후 반환
        return handle.Result as Sprite;
    }

    // 2. 스프라이트 형식으로 로드 실행
    AsyncOperationHandle<Sprite> handleOrigin =
Addressables.LoadAssetAsync<Sprite>(address);

    try
    {
        // ToUniTask()를 통해 비동기 대기
        Sprite result = await handleOrigin.ToUniTask();

        // 3. 핸들 저장 (나중에 Release하기 위함)
        _handles[address] = handleOrigin;

        return result;
    }
    catch (System.Exception)
    {
        // 4. 로드 실패 시 처리
        Debug.LogError($"스프라이트 로드 실패: {address}");

        if (handleOrigin.IsValid())
            Addressables.Release(handleOrigin);

        return null;
    }
}

```

```

// 3. 메모리 해제 함수 (중요!)
public void Release(string address)
{
    if (_handles.TryGetValue(address, out AsyncOperationHandle handle))
    {
        Addressables.Release(handle);
        _handles.Remove(address);
        Debug.Log($"에셋 메모리 해제 완료: {address}");
    }
}
}

```

```

using UnityEngine;

public class DaniTechSoundManager : MonoBehaviour
{
    [SerializeField] private AudioSource AudioSourcePlayer; // 효과음용
    [SerializeField] private AudioSource BGMSourcePlayer; // 배경음용

    public static DaniTechSoundManager Inst { get; set; }

    private void Awake()
    {
        Inst = this;
    }

    public string GetSoundPath(string soundDataId)
    {
        string path = soundDataId;
        // 여기서 데이터 매니저를 통해 사운드 Id로
        // 실제 사운드 데이터 경로를 받아오면 좋다
        return path;
    }

    // 효과음 재생 (겹쳐서 재생 가능)
    public void PlaySFX(string soundDataId)
    {
        DaniTechGameUtil.LoadAndPlayAudioClip(AudioSourcePlayer,
        soundDataId).Forget();
    }

    // 배경음 재생 (교체 재생)

```

```

        public void PlayBGM(string soundDataId)
        {
            DaniTechGameUtil.LoadAndPlayAudioClip(BGMSourcePlayer, soundDataId,
isLoop:true).Forget();
        }

        public void StopBGM()
        {
            BGMSourcePlayer.Stop();
        }

        public void StopSFX()
        {
            AudioSourcePlayer.Stop();
        }
    }
}

```

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class DaniTechUIManager : MonoBehaviour
{
    [SerializeField] Canvas Canvas_BgRoot;
    [SerializeField] Canvas Canvas_MainRoot;
    [SerializeField] Canvas Canvas_ContentRoot;
    [SerializeField] Canvas Canvas_PopupRoot;
    [SerializeField] Canvas Canvas_VeryFrontRoot;

    public static DaniTechUIManager Instance { get; set; }

    // 애는 생성과 제거에 관한 부분 -> Instancing과 가비지컬렉터와 연관이 있는 애
    private Dictionary<DaniTechUIType, DaniTechUIBase> _createdUIDic = new
Dictionary<DaniTechUIType, DaniTechUIBase>();
    // 애는 활성화와 비활성에 관한 부분 -> SetActive
    private HashSet<DaniTechUIType> _openedUIDic = new HashSet<DaniTechUIType>
();

    private void Awake()
    {
        Instance = this;
    }
}

```

```

private void Start()
{
    // 매니저들이 탄생한 후에 UI매니저가 처음으로 게임이 실행될 때 필요한 UI들을 오픈해준다!
    this.ShowStartupUIOnGameStart();
}

public DaniTechUIBase OpenUI(DaniTechUIRootType uiRootType, DaniTechUIType uiType, bool isInitialHide = false)
{
    // 딱히 요청이 있진 않고 오픈만 하면 되는 UI에서 사용

    var openedUI = GetCreatedUI(uiRootType, uiType);

    bool isSetActiveOnOpen = (isInitialHide == false); // 열었을 때 기본적으로 숨겨서 열 것인지 체크
    if (_openedUIDic.Contains(uiType) == false)
    {
        openedUI.gameObject.SetActive(isSetActiveOnOpen);
        _openedUIDic.Add(uiType);
    }

    return openedUI;
}

public void CloseUI(DaniTechUIRootType uiRootType, DaniTechUIType uiType)
{
    if (_openedUIDic.Contains(uiType))
    {
        var openedUi = _createdUIDic[uiType];
        openedUi.gameObject.SetActive(false);
        _openedUIDic.Remove(uiType);
    }
}

private Transform GetRootTransform(DaniTechUIRootType uiRootType)
{
    Transform root = null;
    switch (uiRootType)
    {
        case DaniTechUIRootType.BackgroundUI:
            root = Canvas_BgRoot.transform;
            break;
        case DaniTechUIRootType.MainUI:
            root = Canvas_MainRoot.transform;

```

```

        break;
    case DaniTechUIRootType.ContentUI:
        root = Canvas_ContentRoot.transform;
        break;
    case DaniTechUIRootType.PopupUI:
        root = Canvas_PopupRoot.transform;
        break;
    case DaniTechUIRootType.VeryFrontUI:
        root = Canvas_VeryFrontRoot.transform;
        break;
    }
    return root;
}

private void CreateUI(DaniTechUIRootType uiRootType, DaniTechUIType
uiType)
{
    if (_createdUIDic.ContainsKey(uiType) == false)
    {
        string path = this.GetUIPath(uiRootType, uiType);
        GameObject loadedObj = (GameObject)Resources.Load(path);
        Transform root = GetRootTransform(uiRootType);
        GameObject gObj = Instantiate(loadedObj, root);
        if (gObj != null)
        {
            var uiBase = gObj.GetComponent<DaniTechUIBase>();
            _createdUIDic.Add(uiType, uiBase);
        }
    }
}

private DaniTechUIBase GetCreatedUI(DaniTechUIRootType uiRootType,
DaniTechUIType uiType)
{
    if (_createdUIDic.ContainsKey(uiType) == false)
    {
        CreateUI(uiRootType, uiType);
    }
    return _createdUIDic[uiType];
}

public DaniTechUIBase OpenContentUI(DaniTechUIType uiType)
{
    return OpenUI(DaniTechUIRootType.ContentUI, uiType);
}

```

```

    public DaniTechUIBase OpenPopupUI(DaniTechUIType uiType)
    {
        return OpenUI(DaniTechUIRootType.PopupUI, uiType);
    }

    public void CloseContentUI(DaniTechUIType uiType)
    {
        CloseUI(DaniTechUIRootType.ContentUI, uiType);
    }

    public void ClosePopupUI(DaniTechUIType uiType)
    {
        CloseUI(DaniTechUIRootType.PopupUI, uiType);
    }
}

```

```

using UnityEngine;

public enum DaniTechUIRootType
{
    None = 0,
    BackgroundUI,
    MainUI,
    ContentUI,
    PopupUI,
    VeryFrontUI
}

public enum DaniTechUIType
{
    DNSimplePopup,
    DNMainUI,
    DNMyProfilePopup, // 신규UI추가 1) 새로운 UIType을 추가한다
    DNInventory,
    DNLoadingUI,
    DNDialogueUI,
    DNInfoBookUI
}

public static class DaniTechUIManagerExtension
{
    public static string GetUIPath(this DaniTechUIManager uiManager,

```



```

DaniTechUIRootType uiRootType, DaniTechUIType uiType)
{
    string path = string.Empty; // "" == string.Empty

    // 신규UI추가 2) Resources.Load를 할 경로를 직접 명시한다
    // 해당 경로는 프로젝트창에서 Resources/Prefabs/UI폴더 내에 있는 RootType 폴
더명과 UIType 프리팹 이름과 동일해야 한다! (ex. ContentUI/DNMyProfilePopup)
    path = $"Prefabs/UI/{uiRootType}/{uiType}";
    return path;
}

public static void ShowStartupUIOnGameStart(this DaniTechUIManager
uiManager)
{
    uiManager.OpenLoadingUI();
    uiManager.OpenUI(DaniTechUIRootType.MainUI, DaniTechUIType.DNMainUI);
    // 게임 로비 UI를 여기서 오픈해주자 -> uiManager.
    // MainUI도
}

public static void OpenSimplePopup(this DaniTechUIManager uiManager,
string msg)
{
    var uiBase = uiManager.OpenPopupUI(DaniTechUIType.DNSimplePopup);
    if (uiBase == null)
    {
        Debug.LogWarning($"UI가 생성되지 않았습니다");
        return;
    }

    if (uiBase is DaniTech_SimplePopup simplePopup)
    {
        simplePopup.SetUI(msg);
    }
}

// 신규UI추가 3) 이렇게 어떤 팝업을 열고, 열때 전달해야하는 파라미터가 있다면 이렇게
전달한다.
// 추가하기 편하게 그냥 빼둔 확장 메서드이므로, uiManager과 this는 우선 넘어가
자

public static void OpenMyProfilePopup(this DaniTechUIManager uiManager,
string characterDataId)
{
    // 신규UI추가 4) 이렇게 UI 타입을 던져서 UI 생성을 요청한다
    var uiBase = uiManager.OpenPopupUI(DaniTechUIType.DNMyProfilePopup);
    if (uiBase == null)

```

```

        {
            Debug.LogWarning($"UI가 생성되지 않았습니다");
            return;
        }

        if (uiBase is DaniTech_MyProfilePopup myProfilePopup)
        {
            myProfilePopup.RefreshCharacterUI(characterDataId);
        }
    }

    public static void OpenInventoryPopup(this DaniTechUIManager uiManger)
    {
        var uiBase = uiManger.OpenContentUI(DaniTechUIType.DNInventory);
        if (uiBase == null)
        {
            Debug.LogWarning($"UI가 생성되지 않았습니다");
            return;
        }
    }

    public static void OpenLoadingUI(this DaniTechUIManager uiManager)
    {
        var uiBase = uiManager.OpenUI(DaniTechUIRootType.VeryFrontUI,
DaniTechUIType.DNLoadingUI);
        if (uiBase == null)
        {
            Debug.LogWarning($"UI가 생성되지 않았습니다");
            return;
        }
    }

    public static void CloseLoadingUI(this DaniTechUIManager uiManager)
    {
        uiManager.CloseUI(DaniTechUIRootType.VeryFrontUI,
DaniTechUIType.DNLoadingUI);
    }

    public static void OpenDialogueUI(this DaniTechUIManager uiManager, string
startDialogueId)
    {
        var uiBase = uiManager.OpenContentUI(DaniTechUIType.DNDDialogueUI);
        if(uiBase == null)
        {
            Debug.LogWarning($"UI가 생성되지 않았습니다");
            return;
        }
    }

```

```

        }

        if (uiBase is DaniTech_DialogueUI dialogueUi)
        {
            dialogueUi.StartDialogue(startDialogueId);
        }
    }
}

```

```

using System;
using System.Collections.Generic;

[System.Serializable]
public class GameDataBase
{
    public string Id;
}

// C# 때와 약간 달라진 점
// System.Text.Json 대신 유니티 내장 JsonUtility를 사용
// 따라서 프로퍼티 말고 그냥 일반 public 멤버변수로 변경함
// [System.Serializable]가 없다면 JsonUtility는 데이터를 무시

[System.Serializable]
public class DNCharacterData : GameDataBase
{
    public string Name;
    public string SkillList;
    public string UseWeaponId;
    public string BasicCostumeId;
}

[System.Serializable]
public class DNSkillData : GameDataBase
{
    public string Name;
    public string Description;
}

[System.Serializable]
public class DNWeaponData : GameDataBase
{
    public string Name;
}

```

```

        public string Description;
    }

[System.Serializable]
public class DNCostumeData : GameDataBase
{
    public string Name;
    public string Description;
}

[System.Serializable]
public class DNItemData : GameDataBase
{
    public string Name;
    public string Description;
    public string ItemType;
    public string Grade;
    public string MaxStackCount;
    public string SellingPrice;
    public string IconPath;
}

[System.Serializable]
public class DNDDialogueGroupData : GameDataBase
{
    public List<string> DialogueIdList;
}

[System.Serializable]
public class DNDDialogueData : GameDataBase
{
    public string CharacterDataId;
    public string Description;
    public string NextDialogueId;
    public List<string> SelectionNameList;
    public List<string> SelectionDialogueIdList;
    public string TexturePath;
    public string VoicePath;
}

[System.Serializable]
public class DNFieldObjectData : GameDataBase
{
    public string Name;
    public string Description;
    public string FieldObjectType;
}

```

```

        public List<int> DropCountRange;
        public string DropItemDataId;
        public string IconPath;
        public string PrefabPath;
    }

[System.Serializable]
public class DNMonsterData : GameDataBase
{
    public string Name;
    public string Description;
    public string IconPath;
    public string PrefabPath;
}

```

```

using System;
using System.Collections.Generic;
using UnityEngine;

// 2-1) 플레이어 데이터에 습득 아이템을 저장하기 위해 미리 만들
[System.Serializable]
public class DaniTechItemModel
{
    public long ItemUniqueId;
    public string ItemDataId;
    public int ItemStackCount;
}

// 1) 플레이어 데이터를 만들어보자
// 1-1) JsonUtility로 직렬화하려면, Mono를 상속받지 않도록 주의하자!
[System.Serializable]
public class DaniTechPlayerModel
{
    public string PlayerName;
    public int PlayerTotalExp;
    public string LastMapDataId;
    public Vector3 LastMapPosition;

    public List<DaniTechItemModel> ItemList = new List<DaniTechItemModel>();
}

```

추가로 UI의 구조는 이런 식으로 많이 쓸거야

-> DaniTechUIBase를 상속 받음

-> UGUI기반

-> 버튼은 UIButton이 따로 래핑 클래스로 감싸져 있음

-> 버튼의 이벤트는 유니티에서 직접 등록하는게 아니라 코드에서 해당 버튼에 만들어둔 **public void BindOnClickButtonEvent(Action onClickCallback)**로 연결

-> UGUI의 프리팹은 만들거지만, 혹시 구현하려는 UI에 필요한 컴포넌트가 있다면 다음 코드를 참고해서 제안해줘

```
using System.Collections.Generic;
using UnityEngine;

// 관리주체 역할
public class DaniTech_SampleInventoryUI : DaniTechUIBase
{
    [SerializeField] private GameObject Prefab_Slot;
    [SerializeField] private Transform Transform_UISlotRoot;
    [SerializeField] private DaniTechUIButton Button_CreateSlot;
    [SerializeField] private DaniTechUIButton Button_CloseSelf;
    [SerializeField] private DaniTechUIButton Button_CloseSelfAllArea;

    private int _generatedKey = 0;
    private Dictionary<int, DaniTech_SampleInventorySlotUI> _itemSlotList =
new Dictionary<int, DaniTech_SampleInventorySlotUI>();

    private void OnEnable()
    {
        Button_CreateSlot.BindOnClickButtonEvent(OnClick_CreateSlotTest);
        Button_CloseSelf.BindOnClickButtonEvent(OnClick_ClosePopup);
        Button_CloseSelfAllArea.BindOnClickButtonEvent(OnClick_ClosePopup);
        SetInventoryItemSlotOnEnable();
    }

    private void SetInventoryItemSlotOnEnable()
    {
        // 슬롯 정리 - 혹시 오픈 시점에 다른 슬롯들이 있다면 제거하자
        if(_itemSlotList.Count > 0)
        {
            foreach(var slot in _itemSlotList){
                DestroyImmediate(slot.Value.gameObject);
            }
            _itemSlotList.Clear();
        }
    }
}
```

//인벤오픈 1-1) 인벤토리가 열릴때 플레이어가 보유한 모든 아이템을 출력하는 로직을 넣어봅시다

```
var itemList = DaniTechGameManager.Inst.GetPlayerItemList();
if(itemList == null || itemList.Count == 0)
{
    Debug.LogWarning("보유한 아이템이 없습니다!");
    return;
}

foreach (var itemModel in itemList)
{
    CreateSlot(itemModel.ItemDataId, itemModel.ItemStackCount);
}

}

private void OnDisable()
{
    // 소멸이니까 나중에 신경써주셔도 되요
    // _itemSlotList.Clear();
    // Destroy
}

public void OnClick_ClosePopup()
{
    DaniTechUIManager.Instance.CloseContentUI(DaniTechUIType.DNInventory);
}

public void OnClick_CreateSlotTest()
{
    // CreateSlot();
}

private void CreateSlot(string itemDataId, int itemStackCount)
{
    // 1-1 수동 SetParant가 뒤에 지금은 자동으로 해주고 있다
    var gObj = Instantiate(Prefab_Slot, Transform_UISlotRoot);
    if (gObj == null) return;

    // 1-2 자식 슬롯의 컴포넌트를 가져온다 -> 위에 게임오브젝트는 스크립트가 아직
    아니므로
    var slotComponent = gObj.GetComponent<DaniTech_SampleInventorySlotUI>
();
    if(slotComponent == null) return;
```

```

        _generatedKey++;

        // 1-3 여기서 slotComponent가지고 뭔가를 하는 겁니다!
        slotComponent.InitSlot(_generatedKey, itemDataId, itemStackCount);
        slotComponent.gameObject.name = $"ItemSlot :
{slotComponent.SlotInstanceId}";

        // 1-4 중복체크 해주면 좋긴 하지만, 일단 쉽게 컴포넌트(컴포넌트로 게임오브젝트는
받을 수 있으므로)를 보관해보자
        _itemSlotList.Add(slotComponent.SlotInstanceId, slotComponent);

        slotComponent.BindSlotSelectEvent(OnChildSlotSelected);
    }

    private void OnChildSlotSelected(int selectedSlotInstanceId)
    {
        foreach(var slotKv in _itemSlotList)
        {
            var slot = slotKv.Value;
            bool isSlotSelected = (selectedSlotInstanceId ==
slot.SlotInstanceId);
            slot.ChangeSelectedState(isSlotSelected);
        }

        Debug.LogWarning($"자식 슬롯 {selectedSlotInstanceId} 선택됨!");
    }
}

```

```

using Cysharp.Threading.Tasks;
using System;
using UnityEngine;
using UnityEngine.UI;
using static TMPro.SpriteAssetUtilities.TexturePacker_JsonArray;

// 1: n으로 n의 관계에 있는 슬롯 (자식)
public class DaniTech_SampleInventorySlotUI : MonoBehaviour
{
    [SerializeField] private Text Text_StackCount;
    [SerializeField] private DaniTechUIButton Button_Slot;
    [SerializeField] private Image Image_Icon;
    [SerializeField] private Image Image_Frame;
    [SerializeField] private Image Image_Selected;
}

```



```

private event Action<int> OnSelectEvent;

public int SlotInstanceId { get; private set; }

private void OnEnable()
{
    Image_Selected.gameObject.SetActive(false);
    Button_Slot.BindOnClickButtonEvent(OnClick_SelectItem);
}

public void SetIcon(string itemDataId, int itemCount)
{
    var itemData =
DaniTechGameDataManager.Instance.GetDNItemData(itemDataId);
    if (itemData == null)
    {
        Debug.LogWarning($"Item 데이터를 불러올 수 없습니다! 경로:
{itemDataId}");
        return;
    }

    string iconPath = itemData.IconPath;
    if (string.IsNullOrEmpty(iconPath) == true)
    {
        Debug.LogWarning($"Item 데이터에 아이콘 경로가 존재하지 않습니다.");
        return;
    }

    // + Addressable을 적용하면서 비동기로 바뀌었다
    //DaniTechResourceManager.Inst.LoadSprite(iconPath, (sprite) => {
    //    Image_Icon.sprite = sprite;
    //});

    DaniTechGameUtil.LoadAndSetSpriteImage(Image_Icon, iconPath).Forget();

    //var sprite = GameUtil.LoadSpriteCanBeNull(iconPath);
    //if(sprite == null)
    //{
    //    Debug.LogWarning($"Sprite를 불러올 수 없습니다! 경로:{iconPath}");
    //    return;
    //}
    //Image_Icon.sprite = sprite;

    Text_StackCount.text = $"{itemCount}";
}

```

```

private void OnDisable()
{
    OnSelectEvent = null;
}

public void InitSlot(int slotInstanceId, string itemDataId, int
itemStackCount)
{
    SlotInstanceId = slotInstanceId;
    SetIcon(itemDataId, itemStackCount);
    // Text_StackCount.text = slotInstanceId.ToString();
}

public void OnClick_SelectItem()
{
    // 부모한테 알려주자
    OnSelectEvent?.Invoke(SlotInstanceId);

    Debug.Log($"{SlotInstanceId}눌러졌다");
    // 나중에 툴팁, 팝업 다 여기서 띄워주면 된다
}

public void BindSlotSelectEvent(Action<int> onSelectEvent)
{
    // 애는 부모 하나만 콜백이벤트 등록하면 된다.
    OnSelectEvent = onSelectEvent;
}

// 수동태로 자신의 상태UI를 변경합니다
public void ChangeSelectedState(bool isSelected)
{
    Image_Selected.gameObject.SetActive(isSelected);
}
}

```

```

using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;

public class DaniTech_DialogueUI : DaniTechUIBase
{

```

```

[SerializeField] private GameObject Layout_CharacterName;
[SerializeField] private Text Text_Character;
[SerializeField] private Text Text_Description;
[SerializeField] private DaniTechUIButton Button_Next;

private string _currentDialogueId;
private Queue<string> _descriptionQueue = new Queue<string>();

private void OnEnable()
{
    Button_Next.BindOnClickButtonEvent(OnClick_Next);
}

// 다이얼로그에서 Next 버튼이 눌러질때 호출된다
public void OnClick_Next()
{
    // 다음 대사가 있는지 체크한다
    bool isNextDescriptionExist = CheckAndSetDescription();

    if (isNextDescriptionExist)
    {
        return;
    }

    // 대사가 없다면, 다음으로 이어지는 다이얼로그가 있는지 체크한다
    bool isNextDialogueExist = CheckAndStartNextDialogue();
    if(isNextDialogueExist == false)
    {
        DaniTechUIManager.Instance.CloseContentUI(DaniTechUIType.DNDDialogueUI);
    }

    private bool CheckAndStartNextDialogue()
    {
        var dialogueData =
        DaniTechGameDataManager.Instance.GetDNDDialogueData(_currentDialogueId);
        if (dialogueData == null)
        {
            Debug.LogWarning($"다이얼로그 데이터가 존재하지 않습니다
{dialogueData}");
            return false;
        }

        // 현재 데이터를 기준으로 다음 다이얼로그가 있는지 체크해보고, 있다면 다음 다이
        // 어로그를 시작한다!

```

```

        string nextDialogueId = dialogueData.NextDialogueId;
        if (string.IsNullOrEmpty(nextDialogueId) == false)
        {
            StartDialogue(nextDialogueId);
            return true;
        }

        return false;
    }

    // 다이얼로그를 시작하는 메서드 (외부에서 UIManager를 통해 다이얼로그 시작을 요청할
    때도 쓴다!)
    public void StartDialogue(string dialogueId)
    {
        var dialogueData =
DaniTechGameDataManager.Instance.GetDNDDialogueData(dialogueId);
        if (dialogueData == null)
        {
            Debug.LogWarning($"다이얼로그 데이터가 존재하지 않습니다
{dialogueData}");
            return;
        }

        // 현재 진행중인 다이얼로그 Id는 다음 다이얼로그가 있는지 체크할 때 쓸 수 있도록
        보관한다
        _currentDialogueId = dialogueId;

        // 혹시 현재 대사가 너무 길거나 다음 페이지 처리가 필요할 때 <np> 키워드로 잘라
        주자!
        if (dialogueData.Description.Contains("<np>"))
        {
            string[] dialogueDescriptionList =
dialogueData.Description.Split("<np>");
            foreach (string desc in dialogueDescriptionList)
            {
                _descriptionQueue.Enqueue(desc);
            }
            CheckAndSetDescription();
        }
        else
        {
            // Np 태그가 없다면 바로 다이얼로그 UI를 세팅하자
            SetCurrentDialogueDescription(dialogueData.Description);
        }

        SetCharacterName(dialogueData.CharacterDataId);
    }

```

```

    }

    private bool CheckAndSetDescription()
    {
        bool isNextDescriptionExsist = (_descriptionQueue.Count > 0);
        if (isNextDescriptionExsist)
        {
            string desc = _descriptionQueue.Dequeue();
            SetCurrentDialogueDescription(desc);
        }

        return isNextDescriptionExsist;
    }

    private void SetCharacterName(string characterDataId)
    {
        // 캐릭터 정보가 있다면 말하는 이의 추가 정보를 표기해줄 수 있도록 연동하는 부분
        bool isActive = (string.IsNullOrEmpty(characterDataId) == false);
        Layout_CharacterName.SetActive(isActive);

        if (isActive)
        {
            var characterData =
DaniTechGameDataManager.Instance.GetCharacterData(characterDataId);
            if(characterData != null)
            {
                Text_Character.text = characterData.Name;
            }
        }
    }

    private void SetCurrentDialogueDescription(string description)
    {
        Text_Description.text = description;
    }
}

```

```

using UnityEngine;

public class DaniTechUIBase : MonoBehaviour
{
}

```

```

using System;
using UnityEngine;
using UnityEngine.UI;

public class DaniTechUIButton : MonoBehaviour
{
    [SerializeField] private Button Button_Base;
    [SerializeField] private Text Text_Base;
    [SerializeField] private Image Image_Base;
    [SerializeField] private Image Image_Select;

    private void Awake()
    {
        // 1-2) 이 오브젝트가 생성될 때, 한번 컴포넌트를 찾아서 캐싱하자
        InitUIButton();
        SetDefaultUI();
    }

    private void OnEnable()
    {
        BindOnClickButtonEvent(OnClickSetSelectUI);
    }

    private void OnDisable()
    {
        Button_Base.onClick.RemoveAllListeners();
    }

    private void SetDefaultUI()
    {
        if(Image_Select != null)
        {
            Image_Select.gameObject.SetActive(false);
        }
    }

    private void InitUIButton()
    {
        if(Button_Base != null)
        {
            return;
        }

        // 1-1) 외부에서도 등록할 수 있고,
        // 누군가 누락했다면 등록안해도 알아서 찾아주도록 로직을 넣어 놨다
    }
}

```

```
        var button = this.gameObject.GetComponentInChildren<Button>();
        if(button != null)
        {
            this.Button_Base = button;
        }
    }

    public void BindOnClickButtonEvent(Action onClickCallback)
    {
        if(Button_Base == null) return;

        Button_Base.onClick.AddListener(new
        UnityEngine.Events.UnityAction(onClickCallback));
    }

    public void UnBindOnClickButtonEvent(Action onClickCallback)
    {
        if (Button_Base == null) return;

        Button_Base.onClick.RemoveListener(new
        UnityEngine.Events.UnityAction(onClickCallback));
    }

    public void ChangeButtonText(string buttonStr)
    {
        // 혹시 이버튼을 동적으로, 코드에서 텍스트를 수정해야할 때 사용
        if (Text_Base == null) return;

        Text_Base.text = buttonStr;
    }

    private void OnClickSetSelectUI()
    {
        if(Image_Select != null)
        {
            bool currentActive = Image_Select.gameObject.activeSelf;
            Image_Select.gameObject.SetActive(!currentActive);
        }
    }
}
```



데이터 관리 규칙으로는

static데이터 -> 기획자가 미리 작성해야하고 게임시간에 변할 데이터가 아니라면 엑셀로 작성할거고, 엑셀에는 유니티 기본자료형과 일치하도록 되어 있어, 대신 **List<T>**만 **string[]**이나 **int[]** 같은 방식으로 스키마를 구성하는데 컨버팅된게 유니티에서 가져올땐 **List<T>**로 될거야 (**GameData**) 클래스 참고

그리고 게임시간에 실시간으로 변해야하고, 이게 저장되어야한다면 이걸 **Data**라는 명명규칙이 아니라 **Model**이라는 명명규칙으로 쓸거야 (**Record**데이터와 인스턴스 데이터를 분리하기 위함) 이 **Model**은 위에 **DaniTechPlayerModel**을 참고해줘

나는 아직 처음이니까 내가 구현하려는 콘텐츠의 데이터가 인스턴스 데이터가 원지, 스택 데이터가 원지 구별해서 설명해줘 (ex. **Card**가 있으면 **Card**의 **Name**, **Description**은 **staticData**인데 **Card**의 강화**Level**이나 현재 보유 버프는 인스턴스 데이터인 것처럼)